

Development

Key techniques used

#	Development Technique
1	Recursion
2	Search algorithms
3	Event Handling
4	Anonymous functions
5	Singleton Class design pattern
6	Complex Data Structures
7	Edit Distance Algorithm
8	String Manipulation
9	Object Oriented Programming (OOP)

Note

The name of the file where the screenshotted code can be found is located at the top right corner of each screenshot.

Method – autoAllocate

The autoAllocate method is called when a Task is added without a specified start time.

Recursion is utilised to reuse existing code while allowing the method to be more readable. The method returns int values for error handling, with 0 representing a successful allocation of the Task. In accordance with success criterion 5, if a Task has to be split up (fragmented) into smaller Tasks, the method calls the fragsFormatter method to obtain an array of durations in minutes that the Task should be split into, which guarantees that all of its fragments can be added.

```
1 2
3 4
5 6
7 8
9 10
11 12
13 14
15 16
17 18
19 20
21 22
23 24
25 26
27 28
29 30
31 32
33 34
35 36
37 38
39 40
41 42
43 44
45 46
47 48
49 50
51 52
53 54
55 56
57 58
59 60
61 62
```

TaskHandler.java | main | 04:54 PM

Annotation 1: CURRENT_TIME is assigned as a constant and used throughout the method to eliminate computational time changing the value of CURRENT_TIME

Annotation 2: Adds the Task to the detected time on the user's machine.

Annotation 3: Adds the Task in between existing Tasks.

Annotation 4: Return statements of 0 are used to ensure that the Task is only added once.

Annotation 5: Adds the Task such that it will now be the last Task out of all Tasks that were already added

Annotation 6: Method fragsFormatter is primarily an arithmetic calculation and can technically be independent from the current context of Task objects.

Annotation 7: Method calls itself again, with a limit to one recursive iteration to not have fragments of fragments, in accordance with client's request in appendix C.

Method – fragsFormatter

This method attempts to solve the mathematical problem of distributing one, large duration, in minutes, into smaller different durations, such that the output array of int values would each be less than at least one value in the collection of freeTimes. This method is isolated from the context of Tasks, as it deals only with durations (elements of real numbers). To do so, this method includes **searching** for minimum numbers, **anonymous functions** for better code readability and **complex data structures** such as ArrayLists.

```
1 2
3 4
5 6
7 8
9 10
11 12
13 14
15 16
17 18
19 20
21 22
23 24
25 26
27 28
29 30
31 32
33 34
35 36
37 38
39 40
41 42
43 44
45 46
47 48
49 50
51 52
53 54
55 56
57 58
59 60
61 62
63 64
65 66
67 68
69 70
71 72
73 74
```

TaskHandler.java | main | 04:06 PM

268 private static int[] fragsFormatter(int taskDuration, ArrayList<Integer> freeTimes) {
 if (taskDuration <= 55)
 return new int[0];

 int maxFrgs = -1;
 // client defined these constants to decide max no. of frags for
 // certain durations
 final int[] TIME_LIMITS = {55, 150, 210};
 final int[] MAX_FRAGS = {2, 3, 4};
 for (int i = 0; i < TIME_LIMITS.length - 1; i++) {
 if (maxFrgs == -1 && TIME_LIMITS[i] < taskDuration
 && taskDuration <= TIME_LIMITS[i + 1])
 maxFrgs = MAX_FRAGS[i];
 }
 if (maxFrgs == -1)
 maxFrgs = MAX_FRAGS[MAX_FRAGS.length - 1];

 // get minimum no. of possible frags
 int duration = taskDuration;
 int minFrgs = 0;
 while (duration > 0 && minFrgs < freeTimes.size()) {
 duration -= freeTimes.get(minFrgs++).intValue();
 }

 if (duration > 0 || minFrgs > maxFrgs) // not enough time in each free time
 return new int[0];

 int[] fragTemplate = new int[minFrgs];
 final int MIN_FRAG_DURATION = 30; // client defined this
 duration = taskDuration; // reset the duration
 // populate fragTemplate greedily
 int freeTimePos = 0;
 for (int i = 0; i < fragTemplate.length && freeTimePos < freeTimes.size(); i++) {
 int freeTime = freeTimes.get(freeTimePos++).intValue();
 if (freeTime >= MIN_FRAG_DURATION && duration > freeTime) {
 fragTemplate[i] = freeTime;
 duration -= freeTime;
 } else if (freeTime >= MIN_FRAG_DURATION) {
 fragTemplate[i] = duration;
 duration = 0;
 } else {
 --i;
 }
 }

 if (duration == 0)
 return fragTemplate;

 // attempt to balance out fragTemplate values
 for (int i = minFrgs; i <= maxFrgs; i++) { // from min frags to max frags allowed
 ArrayList<Integer> modFreeTimes = new ArrayList<>();
 // make a shallow copy of free
 freeTimes.forEach((d) -> modFreeTimes.add(Integer.valueOf(d.intValue())));
 // make all durations a multiple of 5 or 10
 fragTemplate = makeFiveTenMultiple(i, taskDuration);

 int templatePos = 0;
 for (int j = 0; j < modFreeTimes.size() && templatePos < fragTemplate.length; j++) {
 // get free time at j
 int currentFree = modFreeTimes.get(j).intValue();
 if (currentFree >= fragTemplate[templatePos]) { // if free time more than frag time
 // occupy free time and repeat without
 // incrementing j, increment templatePos
 modFreeTimes.set(j--, Integer.valueOf(currentFree - fragTemplate[templatePos]));
 templatePos++;
 }
 }
 }

 if (templatePos == fragTemplate.length)
 return fragTemplate;
 // if all methods of task fragmentation fails
 return new int[0];
}

freeTimes is a collection of Integer objects containing the durations of time in between existing Tasks.

The duration, in minutes, of the Task that needs to be split up

Client-defined constants as per requested in Appendix C

Greedy algorithm is attempted to split taskDuration within the first largest Integer values in freeTimes

If the greedy algorithm fails because it has split the Task into too small of a fragment, it will move on to the next algorithm which tries to distribute it evenly

Since freeTime values will be modified when trying to fit the Task, a separate copy is instantiated and used for this part to prevent making unintentional changes.

In accordance with appendix D, the fragments of the Task would mostly be multiples of 5 or 10

Attempting to find freeTimes to fit the fragments into. If unable, the Task will be split into more fragments until maxFrgs.

Returns if it is possible to fit all fragments within freeTimes

Method – getNextOccurringTask

A **binary search** was used to ensure a logarithmic order of growth so as to allow the program to perform better in the event of a user having a large number of Tasks for a particular day. This method gets the next occurring Task, strictly, so the current occurring Task (if any) would not be counted as the next occurring Task; this method will not return it.



```
1 public static Task getNextOccurringTask(LocalTime currentTime) {
151     if (scheduledTasks.isEmpty())
1         return null;
2
3     int pastPos = 0;
4     int futurePos = scheduledTasks.size() - 1;
5     // binary search for the next task
6     while (pastPos < futurePos) {
7         int currentPos = (pastPos + futurePos) / 2;
8         Task task = scheduledTasks.get(currentPos);
9         if (!task.isAfter(currentTime)) {
10             pastPos = currentPos + 1; // immediate next, not on-going task
11         } else {
12             futurePos = currentPos;
13         }
14     }
15     // if there is no next task, return null
16     Task nextTask = scheduledTasks.get(pastPos);
17     if (!nextTask.isAfter(currentTime)) {
18         nextTask = null;
19     }
20
21     return nextTask;
22 }
23 }
```

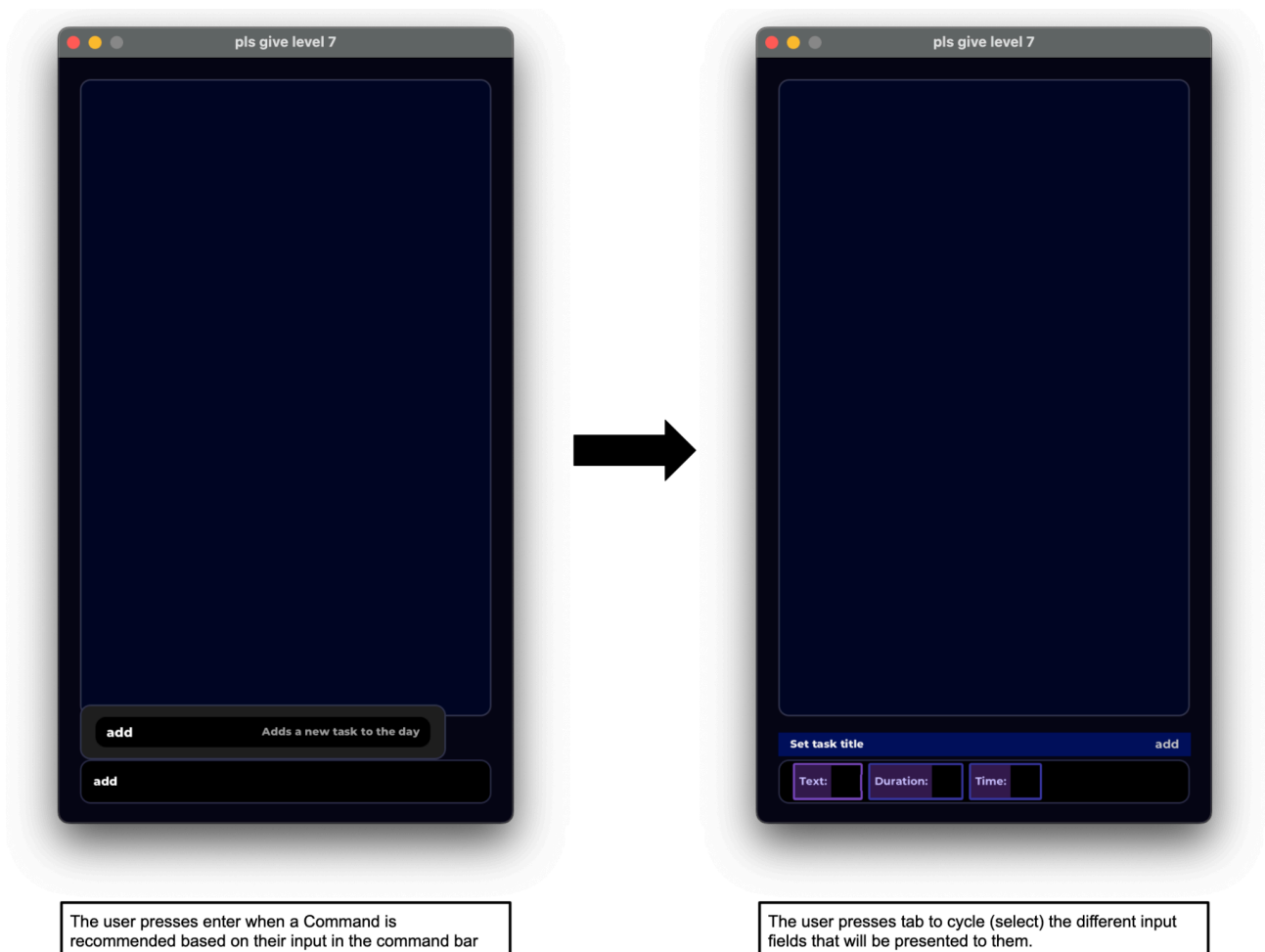
TaskHandler.java | main | 05:33 PM

Condition statement implies: if Task is in the past or currently occurring relative to the current time

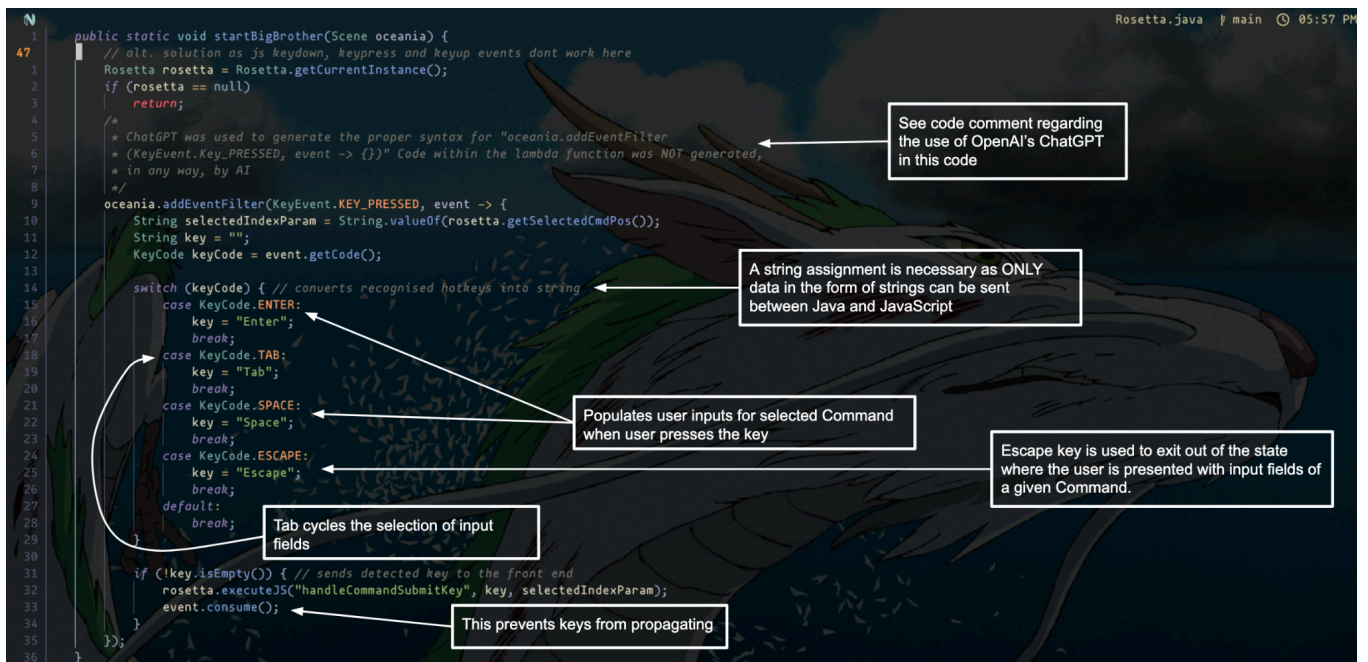
Skip in 1 index ensures that only the next occurring Task is found, not the current occurring Task.

Method – startBigBrother

This method **handles events** by monitoring keyboard inputs in order to detect keys that trigger specific functions in the program, as indicated by the annotations in the image below.



This design decision is justified by success criteria 1 and 9. While JavaScript Web nodes are supposed to have methods for this purpose, it was found, during testing, that the specific keys of keystrokes were not detected by JavaScript or could not be sent to the Java backend. Hence, this method is technically within backend logic:

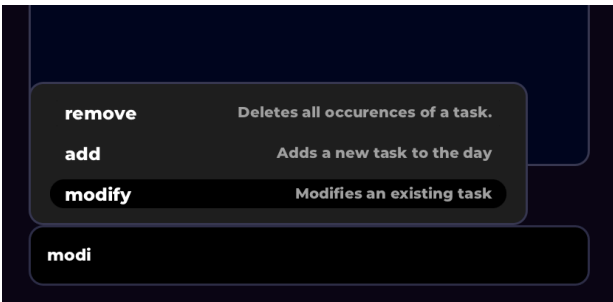
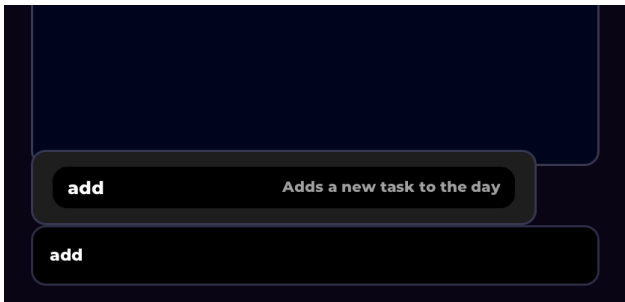


event.consume() is called to prevent the key from executing default behaviour within the UI, such as the default behaviour of the tab key in the web environment. This ensures that such behaviour is practically replaced by the behaviour as specified within this program.

Method – getPotentialCommands

This method features a **selection sort** algorithm which provides the best matches for an incomplete input for the command bar as shown in the UI sketchups in the Internal Assessment (IA) Criterion B document. This is a key component that contributes to the fulfillment of success criteria 1, 8 and 9.



	
List of matches are provided to the user with the closest match being closest to the user input field.	For a perfect match, only one Command is suggested.

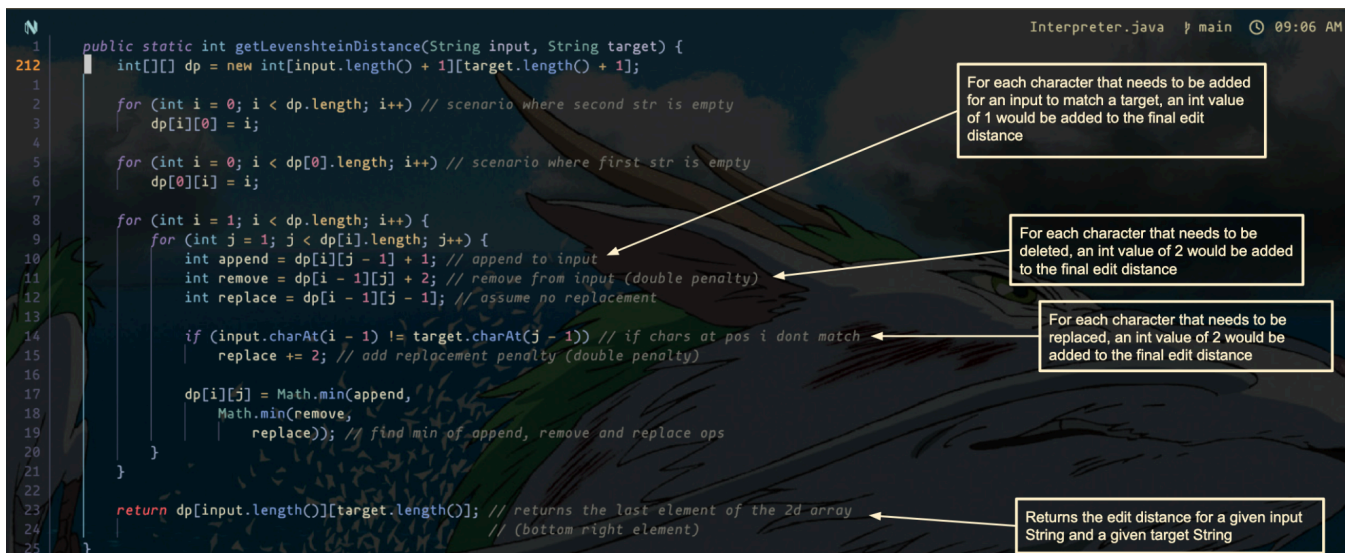
Method – getLevenshteinDistance

This method was adapted from:

<https://www.geeksforgeeks.org/java-program-to-implement-levenshtein-distance-computing-algorithm/>

This implementation of the LevenshteinDistance algorithm outputs an **edit distance** (the spelling similarity) between two Strings as an int value (National Institute of Standards and Technology). The greater the edit distance is, the greater the difference between the two Strings. If the edit distance is 0, the textual content stored in both Strings are the same.

This implementation assigns a lesser penalty to incomplete inputs (inputs that only require additional characters to be matched) compared to inputs that require characters to be removed or replaced given that the method's primary use is regarding the matching of user inputs to a Command, where the input String and target String consists of very few characters (Jain).



Method – parseDuration

This method implements named entity recognition (NER), developed in accordance with the flowchart in the IA Criterion B document, titled “Algorithm Flowchart – Gets duration with Named Entity Recognition” (Belcic). This method, using **string manipulation** techniques, allows a user to enter a String input containing a duration in hours and minutes instead of requiring the user to provide such inputs in dedicated input fields from the frontend, contributing to the fulfilment of success criteria 1 and 8.

Since “hours” and “minutes” start with “h” and “m” respectively, said pattern can be utilised to ensure that any characters after the “h” or “m” does not affect the identification of preceding digits as minutes or hours. This allows the use of flexible syntax, e.g “1 min” is equivalent to “1 m” and “1 hr” is equivalent to “1 h”. Non-digits in between digits and “h” and “m” in the input String, such as spaces, do not impact the identification of digits either.



Method – parseTime

This method, also implementing a form of NER, uses **string manipulation** to take advantage of patterns in the shorthand expression of time to return a new `LocalTime` object from a `String` inputted by the user (Belcic). Whether the user enters the time (as a string) in shorthand form or in a 12-hour format instead of a 24-hour format (e.g 8pm or 8:00pm or 20:00, respectively), the method uses the number of digits and looks for “pm” and “am” substrings to interpret the number of hours and the number of minutes.

```
1 public static LocalTime parseTime(String rawStringTime) {
37 if (rawStringTime.isBlank())
1 return null;
2
3 String stringInput = rawStringTime.toLowerCase();
4 boolean isPM = stringInput.contains("pm");
5 boolean isAM = stringInput.contains("am");
6 StringBuilder numerics = new StringBuilder();
7 char[] chars = stringInput.toCharArray();
8 // copy all digits to numerics
9 for (char c : chars) {
10 if (Character.isDigit(c))
11 numerics.append(c);
12 }
13 // make numerics length of 4
14 if (numerics.length() == 1) {
15 numerics.insert(0, "0");
16 numerics.append("00");
17 } else if (numerics.length() == 3) {
18 numerics.insert(0, "0");
19 } else if (numerics.length() == 2) {
20 numerics.append("00");
21 }
22
23 if (numerics.length() != 4)
24 return null;
25 // get hours
26 int hour = Integer.parseInt(numerics.substring(0, 2));
27 int minute = Integer.parseInt(numerics.substring(2));
28 // conversions to 24 hour format
29 if (hour != 12 && isPM)
30 hour += 12;
31 else if (hour == 12 && isAM) {
32 hour = 0;
33 }
34
35 return LocalTime.of(hour, minute);
36 }
```

String input is sanitised by making it lower cased

If there is only one digit, it represents the hour so a 0 is inserted before the digit and two 0s are inserted after the digit

If there are only three digits, the last two digits are treated as the minute and the first digit as the hour and a 0 is inserted before the three digits

The insertions of 0 make the digits be strictly represented in the format of HHMM (4 digits)

If there are only two digits, those digits will be treated as the hour and two 0s will be inserted following the two digits

If there are more than 4 digits, time cannot be parsed

The first two digits of `numerics` are treated as the hour given the HHMM format

The last two digits of `numerics` are treated as the minute given the HHMM format

If the input contains substring “pm” and the hour was not 12, then add 12 to the hour to convert it into the 24-hour format

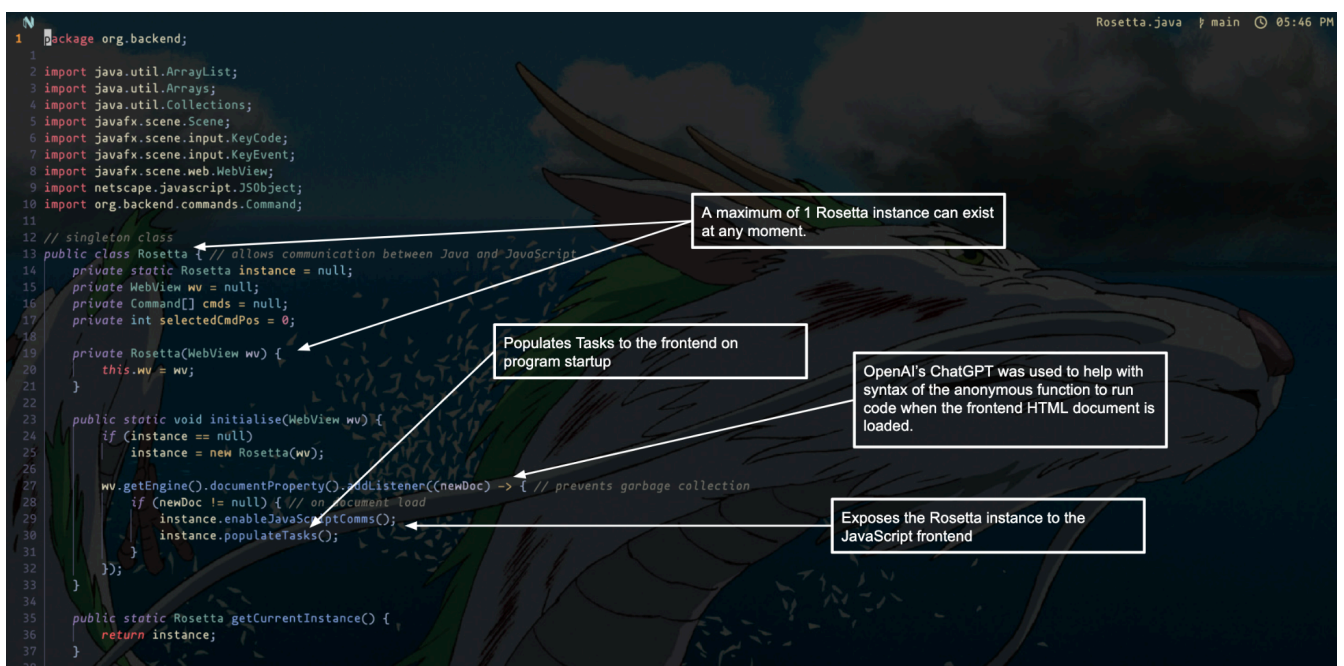
If the input contains substring “am” and the hour was 12, then the hour should be 0 to convert it into the 24-hour format

In the last if statement of the method, hours and minutes expressed in the 12-hour format is converted into the 24-hour format by detecting the presence of the substrings “pm” and “am”. Usually, any number preceded by “pm” requires 12 to be added when converting from the 12-hour format to the 24-hour format (e.g 8pm is equivalent to 20:00). But “12pm” is an exception to the rule since 12pm is equivalent to 12:00. A similar explanation applies to “12am”, where it is expressed as “00:00” in the 24-hour format.

The `parseTime` method contributes to the fulfillment of success criterion 8 and has been developed with reference to the flowchart in IA Criterion B titled “Algorithm Flowchart – Gets time with Named Entity Recognition” (Belcic).

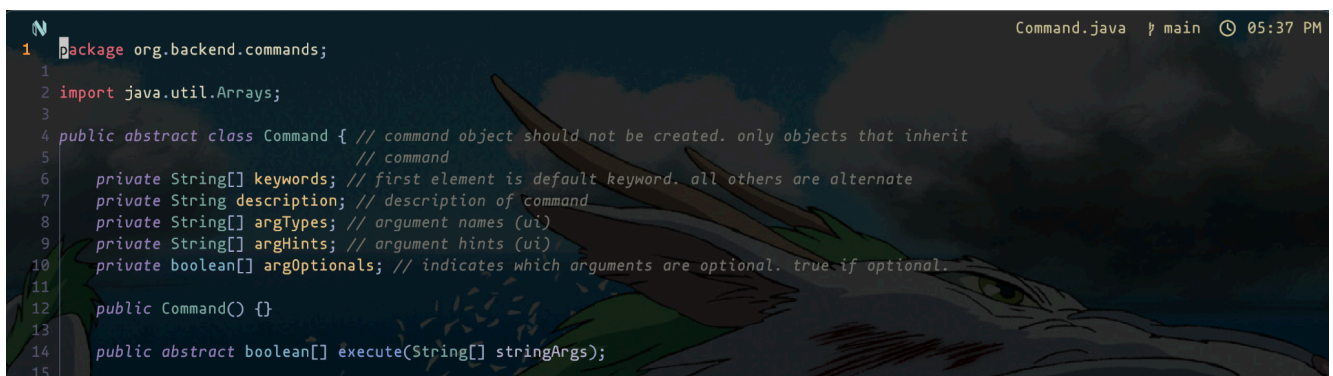
Class – Rosetta

This is a **singleton class** which exposes the Java backend to the JavaScript frontend. Given that only one bridge can exist at any time, the singleton design pattern only allows 1 instance of Rosetta to exist at any time (baeldung). This is an example of **encapsulation** in **OOP** since the single instance of the class can only be accessed using the `getCurrentInstance` method.



Class – Command

This class utilises **OOP** in the form of the abstract keyword in Java to **encapsulate** the creation of Command objects. The keyword prevents a Command object from being instantiated directly from the Command class. Instead, only objects of classes that inherit the Command class can be instantiated (and are objects of the Command data type). This is done to mandate the creation of a new class which extends Command if a new command were to be created in the future, allowing the backend code to be more understandable and intuitive.



```
1 package org.backend.commands;
2 import java.util.Arrays;
3
4 public abstract class Command { // command object should not be created. only objects that inherit
5     // command
6     private String[] keywords; // first element is default keyword. all others are alternate
7     private String description; // description of command
8     private String[] argTypes; // argument names (ui)
9     private String[] argHints; // argument hints (ui)
10    private boolean[] argOptionals; // indicates which arguments are optional. true if optional.
11
12    public Command() {}
13
14    public abstract boolean[] execute(String[] stringArgs);
15 }
```

The Command class also has an execute method with the abstract keyword since each child class of Command (e.g AddCMD) runs a different set of operations. However, each Command still uses the same parameters and returns the same data type. Hence, the use of the abstract keyword mandates that all classes that inherit Command must be executable (implement the execute method).

Works Cited

baeldung. *Singletons in Java*. 16 Dec. 2024, www.baeldung.com/java-singleton. Accessed 1 June 2025.

Belcic, Ivan, and Cole Stryker. "What Is Natural Language Understanding (NLU)?" *IBM*, 3 Mar. 2025, www.ibm.com/think/topics/natural-language-understanding. Accessed 2 June 2025.

Jain, Deep. "How to Calculate Levenshtein Distance in Java?" *Baeldung*, 8 Jan. 2024, www.baeldung.com/java-levenshtein-distance. Accessed 1 June 2025.

National Institute of Standards and Technology. "Levenshtein Distance." *NIST*, 15 May 2019, xlinux.nist.gov/dads/HTML/Levenshtein.html. Accessed 1 June 2025.